

Práctica de Laboratorio: ERP Contable con Vue + Vuetify (JavaScript)

Duración: 4 horas (2 sesiones de 2h)

Objetivo: Construir un MVP funcional del Sistema ERP descrito en los lineamientos, con curva de aprendizaje baja usando JavaScript.

ESTRUCTURA DE LAS SESIONES

Sesión	Duración	Contenido	Módulos ERP
Sesión 1	2 horas	Setup + Layout + CRUD Clientes	Configuración, Clientes/Proveedores
Sesión 2	2 horas	Facturación + Motor Contable + Dashboard	Operaciones, Motor Contable, Reportes

SESIÓN 1: Estructura Base + Módulo Clientes (2 horas)

PASO 1: Crear el proyecto base (15 min)

 Componentes/tecnologías seleccionadas:

- **Vite** → Bundler rápido (viene por defecto)
- **Vue 3** → Framework reactivo
- **Vuetify 3** → Librería Material Design
- **Vue Router 4** → Navegación SPA
- **Material Design Icons (mdi)** → Iconografía

 Referencias oficiales:

- [Vuetify - Instalación](#)
- [Vue Router - Inicio](#)

 Comandos:

```
# 1. Crear proyecto con Vuetify
npm create vuetify@latest erp-contable

# Durante la instalación selecciona:
# ✓ Project name: erp-contable
# ✓ Use TypeScript? → NO (usaremos JavaScript)
```

```
# ✓ Install dependencies? → YES

# 2. Entrar al proyecto
cd erp-contable

# 3. Instalar Vue Router
npm install vue-router@4

# 4. Instalar iconos Material Design
npm install @mdi/font

# 5. Iniciar servidor de desarrollo
npm run dev
```

 Modificar `src/main.js` para activar iconos:

```
import { createApp } from 'vue'
import App from './App.vue'
import vuetify from './plugins/vuetify'
import '@mdi/font/css/materialdesignicons.css' // ← AGREGAR ESTA LÍNEA

createApp(App).use(vuetify).mount('#app')
```

✓ **Checkpoint:** Debes ver la pantalla de bienvenida de Vuetify en <http://localhost:5173>

PASO 2: Configurar Vue Router (15 min)

 Componentes Vuetify usados:

- **Vue Router** → No es de Vuetify, pero es el estándar para SPA en Vue

 Referencia oficial:

- [Vue Router - Installation](#)

 Crear archivo `src/router/index.js`:

```
import { createRouter, createWebHistory } from 'vue-router'

// Importamos las vistas (páginas)
import Dashboard from '../views/Dashboard.vue'
import Clientes from '../views/Clientes.vue'
import Facturacion from '../views/Facturacion.vue'
import Contabilidad from '../views/Contabilidad.vue'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', name: 'dashboard', component: Dashboard },
```

```

    { path: '/clientes', name: 'clientes', component: Clientes },
    { path: '/facturacion', name: 'facturacion', component: Facturacion },
    { path: '/contabilidad', name: 'contabilidad', component: Contabilidad }
  ]
})

export default router

```

🔧 Registrar el router en `src/main.js`:

```

import { createApp } from 'vue'
import App from './App.vue'
import vuetify from './plugins/vuetify'
import router from './router' // ← AGREGAR
import '@mdi/font/css/materialdesignicons.css'

createApp(App)
  .use(vuetify)
  .use(router) // ← AGREGAR
  .mount('#app')

```

📖 PASO 3: Crear las 4 vistas vacías (10 min)

🌿 Componentes Vuetify usados:

- **Ninguno aún** → Solo estructura HTML básica

Crea la carpeta `src/views/` y dentro estos 4 archivos:

📄 `src/views/Dashboard.vue`

```

<template>
  <div>
    <h1 class="text-h4 mb-4">📊 Dashboard Financiero</h1>
    <p>Indicadores principales (lo llenaremos en Sesión 2)</p>
  </div>
</template>

```

📄 `src/views/Clientes.vue`

```

<template>
  <div>
    <h1 class="text-h4 mb-4">👤 Clientes y Proveedores</h1>
    <p>CRUD de clientes (siguiente paso)</p>
  </div>
</template>

```

 src/views/Facturacion.vue

```
<template>
  <div>
    <h1 class="text-h4 mb-4">📄 Facturación</h1>
    <p>Registro de ventas (Sesión 2)</p>
  </div>
</template>
```

 src/views/Contabilidad.vue

```
<template>
  <div>
    <h1 class="text-h4 mb-4">📊 Motor Contable</h1>
    <p>Asientos de partida doble (Sesión 2)</p>
  </div>
</template>
```

PASO 4: Crear el Layout con Menú Lateral (30 min)

 Componentes Vuetify seleccionados:

Componente	¿Por qué lo usamos?	Documentación
v-app	Contenedor raíz OBLIGATORIO de toda app Vuetify	Application
v-app-bar	Barra superior con título y acciones	App bars
v-app-bar-nav-icon	Ícono hamburguesa para abrir/cerrar menú	Incluido en app-bar
v-app-bar-title	Título centrado en la barra	Incluido en app-bar
v-navigation-drawer	Menú lateral deslizante	Navigation drawers
v-list	Lista de opciones del menú	Lists
v-list-item	Cada opción individual del menú	Incluido en lists
v-main	Área de contenido principal	Application
v-container	Contenedor con márgenes y responsive	Grid
v-chip	Etiqueta pequeña para mostrar info	Chips
v-icon	Íconos Material Design	Icons
v-divider	Línea separadora	Dividers

 Referencias oficiales:

- [Application layout](#)
- [Navigation drawers](#)

- [Lists](#)

🔧 Reemplazar `src/App.vue`:

```
<script setup>
import { ref } from 'vue'
import { useRoute } from 'vue-router'

const route = useRoute()
const drawer = ref(true) // Controla si el menú está abierto

// Array con las opciones del menú
const menuItems = [
  { title: 'Dashboard', icon: 'mdi-view-dashboard', to: '/' },
  { title: 'Clientes', icon: 'mdi-account-group', to: '/clientes' },
  { title: 'Facturación', icon: 'mdi-receipt', to: '/facturacion' },
  { title: 'Contabilidad', icon: 'mdi-calculator', to: '/contabilidad' }
]
</script>

<template>
  <v-app>
    <!-- BARRA SUPERIOR -->
    <v-app-bar color="primary" density="comfortable">
      <v-app-bar-nav-icon @click="drawer = !drawer" />
      <v-app-bar-title>
        <v-icon start>mdi-calculator-variant</v-icon>
        ERP Contable - Microempresa
      </v-app-bar-title>
      <v-spacer />
      <v-chip color="white" variant="outlined">
        <v-icon start>mdi-school</v-icon>
        Universidad
      </v-chip>
    </v-app-bar>

    <!-- MENÚ LATERAL -->
    <v-navigation-drawer v-model="drawer" width="260">
      <v-list nav>
        <v-list-item
          v-for="item in menuItems"
          :key="item.to"
          :to="item.to"
          :prepend-icon="item.icon"
          :title="item.title"
          :active="route.path === item.to"
          color="primary"
        />
      </v-list>

      <template v-slot:append>
        <div class="pa-4">
          <v-divider class="mb-4" />
        </div>
      </template>
    </v-navigation-drawer>
  </v-app>
</template>
```

```

        <div class="text-caption text-grey">
          <v-icon size="small">mdi-information</v-icon>
          Sistema didáctico<br />
          Partida doble + SPA
        </div>
      </div>
    </template>
  </v-navigation-drawer>

  <!-- CONTENIDO PRINCIPAL -->
  <v-main>
    <v-container fluid class="pa-6">
      <router-view />
    </v-container>
  </v-main>
</v-app>
</template>

```

💡 Explicación pedagógica:

- `v-model="drawer"` → **Reactividad dos vías**: el botón cambia `drawer`, y `drawer` controla si el menú se ve
- `:to="item.to"` → Integración con Vue Router (navega al hacer clic)
- `:active="route.path === item.to"` → Resalta la opción activa
- `v-for` → Genera un `v-list-item` por cada elemento del array

✅ **Checkpoint:** Ya tienes la SPA corriendo con menú funcional. Prueba navegar entre las 4 vistas.

📖 PASO 5: Crear componente reutilizable `TarjetaKPI` (15 min)

🧩 Componentes Vuetify seleccionados:

Componente	¿Por qué?	Documentación
<code>v-card</code>	Contenedor con sombra y bordes redondeados	Cards
<code>v-card-text</code>	Área de contenido de la tarjeta	Incluido en cards
<code>v-icon</code>	Ícono grande para representar el KPI	Icons

🎯 Concepto Vue que practicamos: **PROPS** (comunicación padre → hijo)

🔧 Crear `src/components/TarjetaKPI.vue`:

```

<script setup>
// Definimos qué propiedades recibe este componente
// En JavaScript usamos la sintaxis con objeto de configuración
defineProps({
  titulo: {
    type: String,
    required: true
  }
})

```

```

    },
    valor: {
      type: [Number, String],
      required: true
    },
    icono: {
      type: String,
      default: 'mdi-information'
    },
    color: {
      type: String,
      default: 'primary'
    }
  })
</script>

<template>
  <v-card :color="color" variant="tonal">
    <v-card-text class="d-flex align-center">
      <v-icon size="40" class="mr-3">{{ icono }}</v-icon>
      <div>
        <div class="text-h4">{{ valor }}</div>
        <div class="text-caption">{{ titulo }}</div>
      </div>
    </v-card-text>
  </v-card>
</template>

```

💡 Explicación pedagógica:

- `defineProps` → Declara qué datos espera recibir del padre
- `:color="color"` → Los dos puntos `:` indican binding dinámico (usa la variable, no el texto)
- Este componente es **reutilizable**: lo usaremos en Dashboard y Clientes

📖 PASO 6: Módulo Clientes - CRUD Completo (60 min)

🧩 Componentes Vuetify seleccionados:

Componente	¿Por qué?	Documentación
<code>v-row</code> y <code>v-col</code>	Sistema de grid responsive (12 columnas)	Grid
<code>v-card</code>	Contenedor para la tabla y el formulario	Cards
<code>v-card-title</code>	Título de la tarjeta	Incluido en cards
<code>v-card-text</code>	Contenido de la tarjeta	Incluido en cards
<code>v-card-actions</code>	Área de botones en la tarjeta	Incluido en cards
<code>v-data-table</code>	Tabla con búsqueda, orden y paginación	Data tables
<code>v-dialog</code>	Modal para crear/editar	Dialogs

Componente	¿Por qué?	Documentación
<code>v-text-field</code>	Campo de texto con validación	Text fields
<code>v-select</code>	Lista desplegable	Selects
<code>v-btn</code>	Botones de acción	Buttons
<code>v-chip</code>	Etiquetas de estado (Cliente/Proveedor)	Chips
<code>v-spacer</code>	Espacio flexible entre elementos	Spacer
<code>v-form</code>	Contenedor de formulario con validación	Forms

Referencias oficiales:

- [Data tables](#)
- [Dialogs](#)
- [Text fields - Validation](#)

Conceptos Vue que practicamos:

- **ref** → Variables reactivas
- **computed** → Cálculos que se actualizan solos
- **Validaciones** → Reglas en formularios

Reemplazar `src/views/Cientes.vue`:

```
<script setup>
import { ref, computed } from 'vue'
import TarjetaKPI from '../components/TarjetaKPI.vue'

// ===== ESTADO REACTIVO =====
// ref() hace que Vue "observe" estas variables y actualice la vista cuando cambian
const clientes = ref([
  { id: 1, nombre: 'Distribuidora del Norte S.A.', rfc: 'DN0900101ABC', email:
'ventas@norte.mx', telefono: '555-1234', tipo: 'cliente', saldo: 15000 },
  { id: 2, nombre: 'Papelería Central', rfc: 'PCE850515XYZ', email:
'contacto@central.mx', telefono: '555-5678', tipo: 'proveedor', saldo: -3200 },
  { id: 3, nombre: 'Servicios Técnicos López', rfc: 'STL920320DEF', email:
'lopez@servicios.mx', telefono: '555-9012', tipo: 'cliente', saldo: 8500 }
])

const busqueda = ref('')
const dialog = ref(false)
const editando = ref(false)
const formulario = ref({
  id: 0, nombre: '', rfc: '', email: '', telefono: '', tipo: 'cliente', saldo: 0
})

// ===== VALIDACIONES =====
// Cada función recibe el valor y retorna true (válido) o un mensaje de error
const reglas = {
```

```

    requerido: (v) => !!v || 'Campo obligatorio',
    rfc: (v) => /^[A-ZÑ&]{3,4}\d{6}[A-Z0-9]{3}$/.test(v) || 'RFC inválido (ej:
    DN0900101ABC)',
    email: (v) => /.+@.+\..+/.test(v) || 'Email inválido'
  }

  // ===== COMPUTED =====
  // computed() recalcula automáticamente cuando cambian sus dependencias
  const clientesFiltrados = computed(() => {
    const termino = busqueda.value.toLowerCase()
    return clientes.value.filter(c =>
      c.nombre.toLowerCase().includes(termino) ||
      c.rfc.toLowerCase().includes(termino)
    )
  })

  const totalClientes = computed(() => clientes.value.filter(c => c.tipo ===
  'cliente').length)
  const totalProveedores = computed(() => clientes.value.filter(c => c.tipo ===
  'proveedor').length)
  const saldoNeto = computed(() => clientes.value.reduce((acc, c) => acc + c.saldo,
  0))

  // ===== MÉTODOS =====
  const abrirNuevo = () => {
    editando.value = false
    formulario.value = { id: 0, nombre: '', rfc: '', email: '', telefono: '', tipo:
    'cliente', saldo: 0 }
    dialog.value = true
  }

  const abrirEditar = (cliente) => {
    editando.value = true
    formulario.value = { ...cliente } // Copia para no modificar el original
    dialog.value = true
  }

  const guardar = () => {
    if (editando.value) {
      const idx = clientes.value.findIndex(c => c.id === formulario.value.id)
      if (idx >= 0) clientes.value[idx] = { ...formulario.value }
    } else {
      const nuevoId = Math.max(0, ...clientes.value.map(c => c.id)) + 1
      clientes.value.push({ ...formulario.value, id: nuevoId })
    }
    dialog.value = false
  }

  const eliminar = (id) => {
    if (confirm('¿Eliminar este registro?')) {
      clientes.value = clientes.value.filter(c => c.id !== id)
    }
  }
}
</script>

```

```

<template>
  <div>
    <!-- ENCABEZADO CON TARJETAS KPI (reutilizamos el componente) -->
    <v-row class="mb-4">
      <v-col cols="12" md="4">
        <TarjetaKPI
          titulo="Clientes activos"
          :valor="totalClientes"
          icono="mdi-account-group"
          color="primary"
        />
      </v-col>
      <v-col cols="12" md="4">
        <TarjetaKPI
          titulo="Proveedores"
          :valor="totalProveedores"
          icono="mdi-truck-delivery"
          color="secondary"
        />
      </v-col>
      <v-col cols="12" md="4">
        <TarjetaKPI
          titulo="Saldo neto"
          :valor="'$' + saldoNeto.toLocaleString()"
          icono="mdi-cash-multiple"
          color="success"
        />
      </v-col>
    </v-row>

    <!-- TABLA DE CLIENTES -->
    <v-card>
      <v-card-title class="d-flex align-center">
        <span>Catálogo de Clientes y Proveedores</span>
        <v-spacer />
        <v-text-field
          v-model="busqueda"
          prepend-inner-icon="mdi-magnify"
          label="Buscar..."
          variant="outlined"
          density="compact"
          hide-details
          style="max-width: 300px"
          class="mr-3"
        />
        <v-btn color="primary" @click="abrirNuevo">
          <v-icon start>mdi-plus</v-icon>
          Nuevo
        </v-btn>
      </v-card-title>

      <v-data-table
        :headers="[

```

```

        { title: 'Nombre', key: 'nombre' },
        { title: 'RFC', key: 'rfc' },
        { title: 'Email', key: 'email' },
        { title: 'Tipo', key: 'tipo' },
        { title: 'Saldo', key: 'saldo', align: 'end' },
        { title: 'Acciones', key: 'acciones', sortable: false, align: 'center' }
    ]"
    :items="clientesFiltrados"
    :items-per-page="5"
  >
    <!-- Slot personalizado para columna tipo -->
    <template v-slot:item.tipo="{ item }">
      <v-chip :color="item.tipo === 'cliente' ? 'primary' : 'secondary'"
size="small">
        {{ item.tipo === 'cliente' ? 'Cliente' : 'Proveedor' }}
      </v-chip>
    </template>

    <!-- Slot personalizado para columna saldo -->
    <template v-slot:item.saldo="{ item }">
      <span :class="item.saldo >= 0 ? 'text-success' : 'text-error'">
        ${{ item.saldo.toLocaleString() }}
      </span>
    </template>

    <!-- Slot personalizado para acciones -->
    <template v-slot:item.acciones="{ item }">
      <v-btn icon="mdi-pencil" size="small" color="primary" variant="text"
@click="abrirEditar(item)" />
      <v-btn icon="mdi-delete" size="small" color="error" variant="text"
@click="eliminar(item.id)" />
    </template>
  </v-data-table>
</v-card>

<!-- DIÁLOGO PARA CREAR/EDITAR -->
<v-dialog v-model="dialog" max-width="600" persistent>
  <v-card>
    <v-card-title>
      {{ editando ? 'Editar' : 'Nuevo' }} Registro
    </v-card-title>
    <v-card-text>
      <v-form>
        <v-text-field
          v-model="formulario.nombre"
          label="Razón Social"
          :rules="[reglas.requerido]"
          variant="outlined"
          class="mb-2"
        />
        <v-text-field
          v-model="formulario.rfc"
          label="RFC"
          :rules="[reglas.requerido, reglas.rfc]"

```

```

        variant="outlined"
        class="mb-2"
        hint="Ej: DN0900101ABC"
    />
    <v-text-field
        v-model="formulario.email"
        label="Email"
        type="email"
        :rules="[reglas.requerido, reglas.email]"
        variant="outlined"
        class="mb-2"
    />
    <v-text-field
        v-model="formulario.telefono"
        label="Teléfono"
        variant="outlined"
        class="mb-2"
    />
    <v-select
        v-model="formulario.tipo"
        :items="['cliente', 'proveedor']"
        label="Tipo"
        variant="outlined"
        class="mb-2"
    />
    <v-text-field
        v-model.number="formulario.saldo"
        label="Saldo inicial"
        type="number"
        variant="outlined"
        prefix="$"
    />
</v-form>
</v-card-text>
<v-card-actions>
    <v-spacer />
    <v-btn @click="dialog = false">Cancelar</v-btn>
    <v-btn color="primary" @click="guardar">Guardar</v-btn>
</v-card-actions>
</v-card>
</v-dialog>
</div>
</template>

```

💡 Explicación pedagógica clave:

1. ¿Qué es `ref()`?

```

const contador = ref(0)
// En el script: contador.value++
// En el template: {{ contador }} (Vue quita el .value automáticamente)

```

2. ¿Qué es `computed()`?

```
// Se recalcula SOLO cuando cambian sus dependencias
const total = computed(() => clientes.value.length)
```

3. ¿Qué son los slots en `v-data-table`? Los slots nos permiten **personalizar** cómo se muestra una columna específica.

 Ejercicio para estudiantes (últimos 15 min Sesión 1)

Reto: Agregar un campo `direccion` al formulario de clientes y mostrarlo en una columna adicional de la tabla.

 **Pista:** Modifica el array `clientes`, el objeto `formulario`, y agrega una columna en `:headers`.

SESIÓN 2: Facturación + Motor Contable + Dashboard (2 horas)

PASO 7: Vista de Facturación (40 min)

 Componentes Vuetify seleccionados:

Componente	¿Por qué?	Documentación
<code>v-row / v-col</code>	Layout de 2 columnas (formulario + resumen)	Grid
<code>v-select</code>	Seleccionar cliente	Selects
<code>v-divider</code>	Separar secciones	Dividers
<code>v-snackbar</code>	Notificaciones tipo toast	Snackbars
<code>v-list / v-list-item</code>	Historial de facturas	Lists

 Conceptos Vue:

- `computed` → Cálculo automático de subtotal, IVA, total
- `ref` → Array dinámico de conceptos

 Reemplazar `src/views/Facturacion.vue`:

```
<script setup>
import { ref, computed } from 'vue'

// ===== ESTADO =====
const clientesDisponibles = [
```

```

    'Distribuidora del Norte S.A.',
    'Papelería Central',
    'Servicios Técnicos López'
  ]
  const clienteSeleccionado = ref('')
  const fecha = ref(new Date().toISOString().substr(0, 10))
  const estado = ref('pagada')

  const detalle = ref([
    { concepto: '', cantidad: 1, precioUnitario: 0 }
  ])

  const facturas = ref([])
  const snackbar = ref(false)
  const mensajeSnack = ref('')

  // ===== COMPUTED (Reactividad en tiempo real) =====
  // Se recalculan automáticamente al cambiar cantidad, precio o agregar líneas
  const subtotal = computed(() =>
    detalle.value.reduce((acc, d) => acc + (d.cantidad * d.precioUnitario), 0)
  )
  const iva = computed(() => subtotal.value * 0.16)
  const total = computed(() => subtotal.value + iva.value)

  // ===== MÉTODOS =====
  const agregarLinea = () => {
    detalle.value.push({ concepto: '', cantidad: 1, precioUnitario: 0 })
  }

  const eliminarLinea = (idx) => {
    if (detalle.value.length > 1) detalle.value.splice(idx, 1)
  }

  const emitirFactura = () => {
    if (!clienteSeleccionado.value) {
      mensajeSnack.value = '⚠ Selecciona un cliente'
      snackbar.value = true
      return
    }
    if (subtotal.value <= 0) {
      mensajeSnack.value = '⚠ La factura debe tener al menos un concepto'
      snackbar.value = true
      return
    }

    facturas.value.push({
      id: facturas.value.length + 1,
      cliente: clienteSeleccionado.value,
      fecha: fecha.value,
      subtotal: subtotal.value,
      iva: iva.value,
      total: total.value,
      estado: estado.value
    })
  }

```

```

// Reset
clienteSeleccionado.value = ''
detalle.value = [{ concepto: '', cantidad: 1, precioUnitario: 0 }]
mensajeSnack.value = '✅ Factura emitida correctamente'
snackbar.value = true
}
</script>

<template>
  <div>
    <h1 class="text-h4 mb-4">📄 Emisor de Facturas</h1>

    <v-row>
      <!-- FORMULARIO DE FACTURA -->
      <v-col cols="12" md="7">
        <v-card>
          <v-card-title>Datos de la Factura</v-card-title>
          <v-card-text>
            <v-row>
              <v-col cols="12" md="6">
                <v-select
                  v-model="clienteSeleccionado"
                  :items="clientesDisponibles"
                  label="Cliente"
                  variant="outlined"
                  prepend-inner-icon="mdi-account"
                />
              </v-col>
              <v-col cols="12" md="3">
                <v-text-field
                  v-model="fecha"
                  label="Fecha"
                  type="date"
                  variant="outlined"
                />
              </v-col>
              <v-col cols="12" md="3">
                <v-select
                  v-model="estado"
                  :items="[
                    { title: 'Contado', value: 'pagada' },
                    { title: 'Crédito', value: 'credito' }
                  ]"
                  label="Estado de pago"
                  variant="outlined"
                />
              </v-col>
            </v-row>

            <v-divider class="my-4" />

            <!-- DETALLE DE LA FACTURA -->
            <div class="text-subtitle-1 mb-2">Conceptos</div>

```

```

<v-card
  v-for="(linea, idx) in detalle"
  :key="idx"
  variant="outlined"
  class="mb-2 pa-2"
>
  <v-row align="center">
    <v-col cols="12" md="5">
      <v-text-field
        v-model="linea.concepto"
        label="Concepto"
        variant="outlined"
        density="compact"
        hide-details
      />
    </v-col>
    <v-col cols="4" md="2">
      <v-text-field
        v-model.number="linea.cantidad"
        label="Cant."
        type="number"
        variant="outlined"
        density="compact"
        hide-details
      />
    </v-col>
    <v-col cols="4" md="2">
      <v-text-field
        v-model.number="linea.precioUnitario"
        label="P. Unit."
        type="number"
        variant="outlined"
        density="compact"
        hide-details
        prefix="$"
      />
    </v-col>
    <v-col cols="3" md="2">
      <strong class="text-primary">
        ${{ (linea.cantidad * linea.precioUnitario).toFixed(2) }}
      </strong>
    </v-col>
    <v-col cols="1">
      <v-btn
        icon="mdi-delete"
        size="small"
        color="error"
        variant="text"
        @click="eliminarLinea(idx)"
        :disabled="detalle.length === 1"
      />
    </v-col>
  </v-row>
</v-card>

```

```

        <v-btn variant="outlined" color="primary" @click="agregarLinea"
class="mt-2">
        <v-icon start>mdi-plus</v-icon>
        Agregar concepto
    </v-btn>
</v-card-text>
</v-card>
</v-col>

<!-- RESUMEN DE LA FACTURA -->
<v-col cols="12" md="5">
    <v-card color="primary" variant="tonal">
        <v-card-title>
            <v-icon start>mdi-calculator</v-icon>
            Resumen
        </v-card-title>
        <v-card-text>
            <div class="d-flex justify-space-between mb-2">
                <span>Subtotal:</span>
                <strong>${{ subtotal.toFixed(2) }}</strong>
            </div>
            <div class="d-flex justify-space-between mb-2">
                <span>IVA (16%):</span>
                <strong>${{ iva.toFixed(2) }}</strong>
            </div>
            <v-divider class="my-2" />
            <div class="d-flex justify-space-between">
                <span class="text-h6">TOTAL:</span>
                <span class="text-h6">${{ total.toFixed(2) }}</span>
            </div>
        </v-card-text>
        <v-card-actions>
            <v-spacer />
            <v-btn color="primary" size="large" @click="emitirFactura">
                <v-icon start>mdi-check</v-icon>
                Emitir Factura
            </v-btn>
        </v-card-actions>
    </v-card>

    <!-- HISTORIAL -->
    <v-card class="mt-4" v-if="facturas.length > 0">
        <v-card-title>Últimas facturas</v-card-title>
        <v-list>
            <v-list-item v-for="f in facturas" :key="f.id">
                <template v-slot:prepend>
                    <v-icon color="primary">mdi-receipt</v-icon>
                </template>
                <v-list-item-title>{{ f.cliente }}</v-list-item-title>
                <v-list-item-subtitle>{{ f.fecha }}</v-list-item-subtitle>
                <template v-slot:append>
                    <v-chip :color="f.estado === 'pagada' ? 'success' : 'warning'"
size="small">

```

```

        ${{ f.total.toFixed(2) }}
      </v-chip>
    </template>
  </v-list-item>
</v-list>
</v-card>
</v-col>
</v-row>

<v-snackbar v-model="snackbar" :timeout="3000">
  {{ mensajeSnack }}
</v-snackbar>
</div>
</template>

```

PASO 8: Motor Contable - Partida Doble (40 min)

 Componentes Vuetify seleccionados:

Componente	¿Por qué?	Documentación
<code>v-alert</code>	Mostrar validación Debe=Haber	Alerts
<code>v-table</code>	Tabla simple para partidas	Tables
<code>v-expansion-panels</code>	Libro Diario acordeón	Expansion panels
<code>v-expansion-panel</code>	Cada asiento del diario	Incluido en expansion-panels

 Conceptos Vue:

- **Validación de regla de negocio:** Debe == Haber
- `computed` → Sumas automáticas

 Reemplazar `src/views/Contabilidad.vue`:

```

<script setup>
import { ref, computed } from 'vue'

const asientos = ref([])
const fecha = ref(new Date().toISOString().substr(0, 10))
const concepto = ref('')
const partidas = ref([
  { cuenta: '', debe: 0, haber: 0 },
  { cuenta: '', debe: 0, haber: 0 }
])

const cuentas = [
  '1101 - Caja', '1102 - Bancos', '1103 - Clientes', '1104 - Inventarios',
  '2101 - Proveedores', '2102 - IVA por Pagar',
  '4101 - Ventas', '5101 - Costo de Ventas'
]

```

```

]

// ===== VALIDACIÓN CONTABLE =====
const totalDebe = computed(() => partidas.value.reduce((a, p) => a + Number(p.debe),
0))
const totalHaber = computed(() => partidas.value.reduce((a, p) => a +
Number(p.haber), 0))
const diferencia = computed(() => totalDebe.value - totalHaber.value)
const esValido = computed(() =>
  totalDebe.value > 0 && diferencia.value === 0 && concepto.value.trim() !== ''
)

const agregarPartida = () => {
  partidas.value.push({ cuenta: '', debe: 0, haber: 0 })
}

const eliminarPartida = (idx) => {
  if (partidas.value.length > 2) partidas.value.splice(idx, 1)
}

const registrarAsiento = () => {
  if (!esValido.value) return
  asientos.value.push({
    id: asientos.value.length + 1,
    fecha: fecha.value,
    concepto: concepto.value,
    partidas: partidas.value.map(p => ({ ...p })))
  })
  concepto.value = ''
  partidas.value = [
    { cuenta: '', debe: 0, haber: 0 },
    { cuenta: '', debe: 0, haber: 0 }
  ]
}

// Asiento automático de ejemplo
const generarVentaEjemplo = () => {
  concepto.value = 'Venta de mercadería a crédito'
  partidas.value = [
    { cuenta: '1103 - Clientes', debe: 11600, haber: 0 },
    { cuenta: '4101 - Ventas', debe: 0, haber: 10000 },
    { cuenta: '2102 - IVA por Pagar', debe: 0, haber: 1600 }
  ]
}
</script>

<template>
  <div>
    <h1 class="text-h4 mb-4"><img alt="Motor Contable logo" data-bbox="378 864 398 878"/> Motor Contable - Partida Doble</h1>

    <v-alert type="info" variant="tonal" class="mb-4">
      <strong>Principio de partida doble:</strong> Cada transacción afecta al menos
      dos cuentas.
      La suma del DEBE debe ser igual a la suma del HABER.
    </v-alert>
  </div>
</template>

```

```

</v-alert>

<v-card>
  <v-card-title>
    Nuevo Asiento Contable
  </v-card-title>
  <v-spacer />
  <v-btn variant="outlined" color="secondary" @click="generarVentaEjemplo">
    <v-icon start>mdi-auto-fix</v-icon>
    Cargar ejemplo
  </v-btn>
</v-card>

<v-card-text>
  <v-row>
    <v-col cols="12" md="6">
      <v-text-field v-model="fecha" label="Fecha" type="date"
variant="outlined" />
    </v-col>
    <v-col cols="12" md="6">
      <v-text-field v-model="concepto" label="Concepto" variant="outlined" />
    </v-col>
  </v-row>

  <v-divider class="my-4" />

  <!-- TABLA DE PARTIDAS -->
  <v-table>
    <thead>
      <tr>
        <th>Cuenta</th>
        <th class="text-right">Debe ($)</th>
        <th class="text-right">Haber ($)</th>
        <th></th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="(p, idx) in partidas" :key="idx">
        <td>
          <v-select
            v-model="p.cuenta"
            :items="cuentas"
            variant="outlined"
            density="compact"
            hide-details
          />
        </td>
        <td>
          <v-text-field
            v-model.number="p.debe"
            type="number"
            variant="outlined"
            density="compact"
            hide-details
            :disabled="p.haber > 0"
          />
        </td>
      </tr>
    </tbody>
  </v-table>

```

```

        </td>
        <td>
            <v-text-field
                v-model.number="p.haber"
                type="number"
                variant="outlined"
                density="compact"
                hide-details
                :disabled="p.debe > 0"
            />
        </td>
        <td>
            <v-btn icon="mdi-delete" size="small" color="error" variant="text"
@click="eliminarPartida(idX)" />
        </td>
    </tr>
    <tr class="bg-grey-lighten-4">
        <td class="text-right"><strong>TOTALES</strong></td>
        <td class="text-right"><strong>${{ totalDebe.toFixed(2) }}</strong>
    </td>
        <td class="text-right"><strong>${{ totalHaber.toFixed(2) }}</strong>
    </td>
        <td></td>
    </tr>
</tbody>
</v-table>

<v-btn variant="outlined" class="mt-2" @click="agregarPartida">
    <v-icon start>mdi-plus</v-icon>
    Agregar partida
</v-btn>

<!-- VALIDACIÓN -->
<v-alert
    :type="esValido ? 'success' : diferencia === 0 ? 'info' : 'error'"
    variant="tonal"
    class="mt-4"
>
    <template v-if="esValido">✅ Asiento cuadrado - Listo para
registrar</template>
    <template v-else-if="diferencia === 0">⚠️ Ingresa un concepto para
registrar</template>
    <template v-else>❌ Diferencia: ${{ diferencia.toFixed(2) }} - El asiento
NO cuadra</template>
</v-alert>
</v-card-text>
<v-card-actions>
    <v-spacer />
    <v-btn color="primary" size="large" :disabled="!esValido"
@click="registrarAsiento">
        <v-icon start>mdi-content-save</v-icon>
        Registrar Asiento
    </v-btn>
</v-card-actions>

```

```

</v-card>

<!-- LIBRO DIARIO -->
<v-card class="mt-6" v-if="asientos.length > 0">
  <v-card-title>📖 Libro Diario</v-card-title>
  <v-card-text>
    <v-expansion-panels>
      <v-expansion-panel v-for="asiento in asientos" :key="asiento.id">
        <v-expansion-panel-title>
          <v-chip size="small" class="mr-2">#{{ asiento.id }}</v-chip>
          <strong>{{ asiento.concepto }}</strong>
          <v-spacer />
          <span class="text-caption text-grey mr-4">{{ asiento.fecha }}</span>
          <v-chip color="success" size="small">
            ${{ asiento.partidas.reduce((a,p)=>a+Number(p.debe),0).toFixed(2) }}
          </v-chip>
        </v-expansion-panel-title>
        <v-expansion-panel-text>
          <v-table density="compact">
            <thead>
              <tr><th>Cuenta</th><th class="text-right">Debe</th><th
class="text-right">Haber</th></tr>
            </thead>
            <tbody>
              <tr v-for="(p, i) in asiento.partidas" :key="i">
                <td>{{ p.cuenta }}</td>
                <td class="text-right">{{ p.debe > 0 ? '$' + p.debe.toFixed(2) :
'' }}</td>
                <td class="text-right">{{ p.haber > 0 ? '$' + p.haber.toFixed(2)
: '' }}</td>
              </tr>
            </tbody>
          </v-table>
        </v-expansion-panel-text>
      </v-expansion-panel>
    </v-expansion-panels>
  </v-card-text>
</v-card>
</div>
</template>

```

📖 PASO 9: Dashboard con Indicadores (20 min)

🧩 Componentes Vuetify seleccionados:

Componente	¿Por qué?	Documentación
<code>v-progress-linear</code>	Barras de progreso	Progress linear
<code>v-progress-circular</code>	Indicador circular	Progress circular

🌀 Conceptos Vue:

- Reutilización del componente **TarjetaKPI** (creado en Paso 5)

🔧 Reemplazar **src/views/Dashboard.vue**:

```
<script setup>
import { computed } from 'vue'
import TarjetaKPI from '../components/TarjetaKPI.vue'

// Datos simulados (en producción vendrían de los otros módulos)
const ventasDelMes = 125000
const gastosDelMes = 78000
const clientesActivos = 12
const facturasPendientes = 5

const utilidad = computed(() => ventasDelMes - gastosDelMes)
const margen = computed(() => ((utilidad.value / ventasDelMes) * 100).toFixed(1))
</script>

<template>
  <div>
    <h1 class="text-h4 mb-4">📊 Dashboard Financiero</h1>

    <!-- KPIs PRINCIPALES (reutilizamos TarjetaKPI) -->
    <v-row>
      <v-col cols="12" md="3">
        <TarjetaKPI titulo="Ventas del mes" :valor="'$' +
ventasDelMes.toLocaleString()" icono="mdi-cash" color="primary" />
      </v-col>
      <v-col cols="12" md="3">
        <TarjetaKPI titulo="Gastos del mes" :valor="'$' +
gastosDelMes.toLocaleString()" icono="mdi-cart" color="error" />
      </v-col>
      <v-col cols="12" md="3">
        <TarjetaKPI titulo="Utilidad" :valor="'$' + utilidad.toLocaleString()"
icono="mdi-trending-up" color="success" />
      </v-col>
      <v-col cols="12" md="3">
        <TarjetaKPI titulo="Facturas pendientes" :valor="facturasPendientes"
icono="mdi-account-clock" color="warning" />
      </v-col>
    </v-row>

    <!-- GRÁFICOS -->
    <v-row class="mt-4">
      <v-col cols="12" md="6">
        <v-card>
          <v-card-title>Distribución de gastos</v-card-title>
          <v-card-text>
            <div class="mb-3">
              <div class="d-flex justify-space-between mb-1">
                <span>Compras</span><span>45%</span>
              </div>
              <v-progress-linear model-value="45" color="primary" height="20">

```



```
    </v-card>
  </v-col>
</v-row>
</div>
</template>
```



RESUMEN: COMPONENTES VUETIFY UTILIZADOS

#	Componente	Dónde se usa	Documentación
1	<code>v-app</code>	Layout raíz	Application
2	<code>v-app-bar</code>	Barra superior	App bars
3	<code>v-navigation-drawer</code>	Menú lateral	Navigation drawers
4	<code>v-list</code> / <code>v-list-item</code>	Menú y historial	Lists
5	<code>v-card</code>	Contenedores	Cards
6	<code>v-btn</code>	Botones	Buttons
7	<code>v-text-field</code>	Campos de texto	Text fields
8	<code>v-select</code>	Listas desplegables	Selects
9	<code>v-data-table</code>	Tabla de clientes	Data tables
10	<code>v-dialog</code>	Modal crear/editar	Dialogs
11	<code>v-alert</code>	Validación partida doble	Alerts
12	<code>v-chip</code>	Etiquetas	Chips
13	<code>v-snackbar</code>	Notificaciones	Snackbars
14	<code>v-progress-linear</code>	Barras de progreso	Progress linear
15	<code>v-progress-circular</code>	Indicador circular	Progress circular
16	<code>v-expansion-panels</code>	Libro Diario	Expansion panels
17	<code>v-table</code>	Tabla partidas	Tables
18	<code>v-divider</code>	Separadores	Dividers
19	<code>v-row</code> / <code>v-col</code>	Grid	Grid
20	<code>v-icon</code>	Íconos	Icons



ENTREGABLES DE LA PRÁCTICA

Al final de las 4 horas, los estudiantes tendrán:

- ✓ Una SPA con 4 módulos navegables
 - ✓ CRUD completo de clientes con validaciones
 - ✓ Sistema de facturación con cálculo automático de IVA
 - ✓ Motor contable con validación de partida doble (Debe = Haber)
 - ✓ Dashboard con indicadores financieros
 - ✓ Componente reutilizable **TarjetaKPI** con props
 - ✓ Uso de 20 componentes de Vuetify
 - ✓ Código JavaScript limpio y comentado
-

CONSEJOS PARA EL PROFESOR

1. **Proyecta la documentación de Vuetify** mientras explicas cada componente
2. **Usa el botón "Cargar ejemplo"** en el motor contable para mostrar partida doble
3. **Rompe algo a propósito** (ej. quita `.value` de un `ref`) para que vean cómo falla la reactividad
4. **Pregunta constante:** "¿Qué pasaría si...?" para fomentar pensamiento crítico

¿Quieres que te prepare también el **ejercicio específico de Props y Slots** para reforzar esos conceptos en esta práctica?